Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

За седьмой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:
Студент 4 курса
Группы ИИ-26(1)
Прокопюк А.Д.
Проверил:
Никитин С.К.

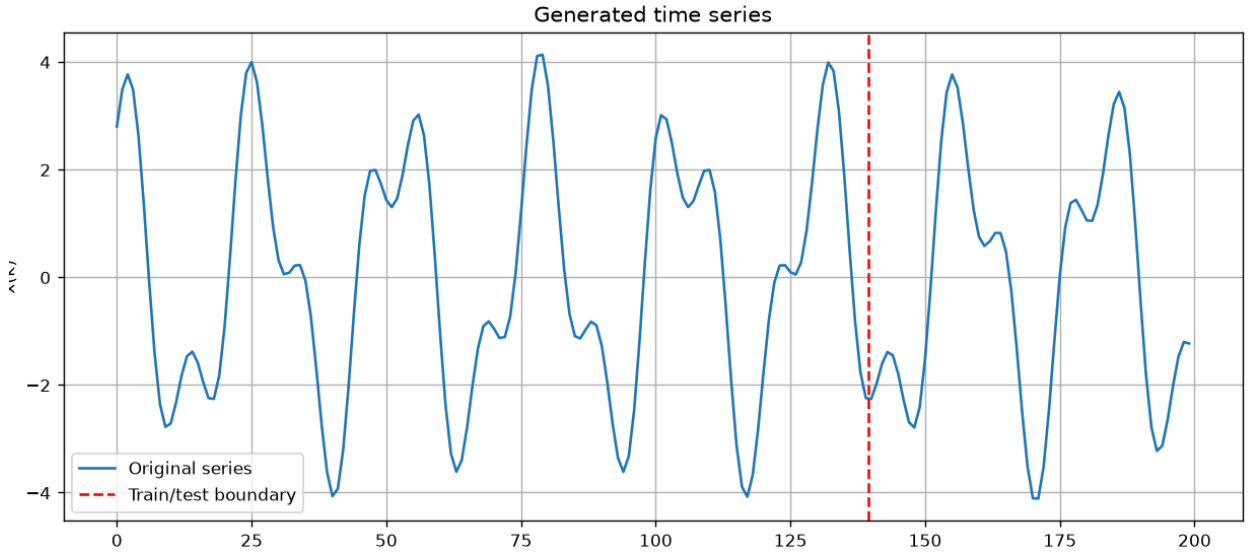
Брест 2026

Цель работы: на основе многослойного персептрона p–N–1 решить задачу прогнозирования временного ряда. Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Вариант 12

Вариант	a	b	c	d	p	N
1	1.0	0.30	0.5	0.70	6	4
2	2.0	0.25	1.0	0.60	8	5
3	1.5	0.40	0.8	0.90	6	5
4	0.8	0.20	0.4	0.50	8	4
5	2.5	0.35	1.2	0.80	7	6
6	1.2	0.28	0.6	0.65	6	4
7	3.0	0.22	1.5	0.55	8	6
8	1.8	0.45	0.9	1.00	6	5
9	0.9	0.33	0.45	0.75	7	4
10	2.2	0.27	1.1	0.62	8	5
11	1.4	0.38	0.7	0.85	6	5
12	2.8	0.24	1.4	0.58	8	6
13	1.1	0.42	0.55	0.95	6	4
14	2.4	0.31	1.2	0.72	7	6
15	1.6	0.26	0.8	0.63	8	5

1. Сгенерировать ряд по формуле варианта и построить его график с отметкой границы между обучающим и тестовым участками.



2.1 Нормализовать ряд по минимуму и максимуму обучающего участка: x_norm = (x - x_min) / (x_max - x_min). Записать формулу обратного пересчёта и использовать её при вычислении всех метрик в исходной шкале. Объяснить, почему min/max нельзя брать по всему ряду.

$$x(k) = x_{norm}(k) * (x_{max} - x_{min}) + x_{min}$$



Потому что тестовая выборка имитирует будущие данные, которые модель ещё не видела. Если использовать её значения для определения минимума и максимума, информация о тестовой выборке попадёт в предварительную обработку обучающих данных. Это называется утечкой данных. В результате оценка качества модели может оказаться менее объективной, поскольку при подготовке использовалась информация из будущего.

2.2 Описание метода: формулы нормализации и обратного пересчёта, схема формирования окон, архитектура p–N–1, вывод градиентов для линейного выхода и сигмоидного скрытого слоя, формулы метрик MAE, RMSE, MAPE/sMAPE.

Формирование обучающих окон. Для прогнозирования используется скользящее окно размера (p=8). Входными данными являются восемь последовательных значений ряда, а целевым значением — следующее значение:

$$X_k = [x_{\text{norm}}(k), \ldots, x_{\text{norm}}(k+p-1)]$$

$$y_k = x_{\text{norm}}(k+p)$$

MAE:

$$\frac{1}{m}\sum_{i=1}^{m}|y_i - e_i|$$

$$RMSE = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(y_i - e_i)^2}$$

$$MAPE = \frac{1}{m}\sum_{i=1}^{m}\left|\frac{y_i - e_i}{y_i}\right|$$

$$sMAPE = \frac{1}{m}\sum_{i=1}^{m}\frac{2|y_i - e_i|}{|y_i| - |e_i|}$$

3. Код программы с комментариями.

```python
import numpy as np

import matplotlib.pyplot as plt

# functions
def linear(wsum):
    return wsum

def der_linear(wsum):
    return np.ones_like(wsum)

def sigmoid(wsum):
    wsum = np.clip(wsum, -500, 500)
    return 1 / (1 + np.exp(-wsum))

def der_sigmoid(wsum):
    y = sigmoid(wsum)
    return y * (1 - y)

def relu(wsum):
    return np.maximum(0, wsum)

def der_relu(wsum):
    return (wsum > 0).astype(float)

# initializations
def init_uniform(shape, fan_in, fan_out):
    return np.random.uniform(-0.1, 0.1, shape)

def init_xavier(shape, fan_in, fan_out):
    limit = np.sqrt(6 / (fan_in + fan_out))
    return np.random.uniform(-limit, limit, shape)

def init_he(shape, fan_in, fan_out):
    limit = np.sqrt(6 / fan_in)
```

```
    return np.random.uniform(-limit, limit, shape)

def init_zero_thresholds(size):
    return np.zeros((1, size))

def init_random_thresholds(size):
    return np.random.uniform(-0.1, 0.1, (1, size))

# metrics
def mse(yj, ej):
    return 0.5 * (yj - ej) ** 2

def calculate_mae(net, X_norm, y_raw, c0=-10, c1=9):
    total_error = 0

    for i in range(len(X_norm)):
        pred_norm = net.forward(X_norm[i:i+1])[0][0]
        pred_raw = denormalize(pred_norm, min_val=c0, max_val=c1)

        total_error += abs(pred_raw - y_raw[i][0])

    return total_error / len(X_norm)


def calculate_metrics(y_true, y_pred):
    mae = np.mean(np.abs(y_true - y_pred))
    rmse = np.sqrt(np.mean((y_true - y_pred) ** 2))

    denominator = np.abs(y_true) + np.abs(y_pred)
    smape_values = np.divide(
        2 * np.abs(y_true - y_pred),
        denominator,
        out=np.zeros_like(denominator, dtype=float),
        where=denominator != 0
    )
    smape = np.mean(smape_values) * 100

    return mae, rmse, smape


def evaluate_network(net, X, y, x_min, x_max):
    predictions_norm = net.forward(X)

    predictions = denormalize(predictions_norm, x_min, x_max)
    actual = denormalize(y, x_min, x_max)

    return calculate_metrics(actual, predictions)


# normaizations/denormalizatins
def normalize(val, min_val=-10, max_val=9):
```

```
    return (val - min_val) / (max_val - min_val)

def denormalize(val, min_val=-10, max_val=9):
    return min_val + val * (max_val - min_val)

class NN:
    def __init__(self, layer_sizes, act_funcs, der_funcs, weight_inits=None,
threshold_inits=None):
        self.layer_sizes = layer_sizes
        self.num_layers = len(layer_sizes)
        self.act_funcs = act_funcs
        self.der_funcs = der_funcs

        if len(act_funcs) != self.num_layers - 1:
            raise ValueError("Each layer needs an activation function")

        if len(der_funcs) != self.num_layers - 1:
            raise ValueError("For each layer, a derivative is needed")

        if weight_inits is not None and len(weight_inits) != self.num_layers - 1:
            raise ValueError("The number of weight initializers does not coincide
with the number of layers")

        if threshold_inits is not None and len(threshold_inits) !=
self.num_layers - 1:
            raise ValueError("The number of threshold initializers does not
coincide with the number of layers")


        if weight_inits is None:
            weight_inits = [init_xavier] * (self.num_layers - 1)

        if threshold_inits is None:
            threshold_inits = [init_zero_thresholds] * (self.num_layers - 1)

        self.weights = []
        self.thresholds = []

        for i in range(self.num_layers - 1):
            fan_in = layer_sizes[i]
            fan_out = layer_sizes[i+1]
            shape = (fan_in, fan_out)

            w = weight_inits[i](shape, fan_in, fan_out)
            T = threshold_inits[i](fan_out)

            self.weights.append(w)
            self.thresholds.append(T)

    def forward(self, X):
        self.wsums = []
```

```
        self.activations = [X]
        current_x = X   #yj

        for i in range(self.num_layers-1):
            wsum = np.dot(current_x, self.weights[i]) - self.thresholds[i]
            self.wsums.append(wsum)
            current_x = self.act_funcs[i](wsum)
            self.activations.append(current_x)

        return current_x

    def backward(self, ethalon, lr):
        error = (self.activations[-1] - ethalon) * self.der_funcs[-
1](self.wsums[-1])
        for i in range(self.num_layers - 2, -1, -1):
            inputs = self.activations[i]
            if i > 0:
                prev_error = np.dot(error, self.weights[i].T) * self.der_funcs[i-
1](self.wsums[i-1])

            self.weights[i] -= np.dot(inputs.T, error) * lr
            self.thresholds[i] += np.sum(error, axis=0, keepdims=True) * lr

            if i > 0:
                error = prev_error

def prepare_data(a, b, c, d, p=8, train_size=140, total_size=200):
    k = np.arange(total_size)
    series = a * np.cos(b * k) + c * np.sin(d * k)

    train_raw = series[:train_size]
    test_raw = series[train_size:]

    x_min = np.min(train_raw)
    x_max = np.max(train_raw)

    series_norm = normalize(series, x_min, x_max)

    X_train, y_train = [], []
    X_test, y_test = [], []

    for i in range(p, train_size):
        X_train.append(series_norm[i-p:i])
        y_train.append(series_norm[i])

    for i in range(train_size, total_size):
        X_test.append(series_norm[i-p:i])
        y_test.append(series_norm[i])

    return (
        np.array(X_train),
```

```
        np.array(y_train).reshape(-1, 1),
        np.array(X_test),
        np.array(y_test).reshape(-1, 1),
        x_min,
        x_max,
        train_raw,
        test_raw
    )

def train_network(net, X_train, y_train, lr=0.1, target_error=1e-4,
max_epochs=20000):
    errors = []
    converged = False

    for epoch in range(max_epochs):
        indices = np.random.permutation(len(X_train))

        for i in indices:
            X = X_train[i:i+1]
            y = y_train[i:i+1]

            net.forward(X)
            net.backward(y, lr)

        predictions = net.forward(X_train)
        error = np.mean((predictions - y_train) ** 2)
        errors.append(error)

        if error <= target_error:
            converged = True
            break

    return net, errors, epoch + 1, converged

def predict_point(net, a, b, c0=-10, c1=9):
    raw_input = np.array([[a, b]])
    norm_input = normalize(raw_input, min_val=c0, max_val=c1)

    pred_norm = net.forward(norm_input)[0][0]
    pred_denorm = denormalize(pred_norm, min_val=c0, max_val=c1)

    threshold_mid = (c0 + c1) / 2
    assigned_class = c1 if pred_denorm >= threshold_mid else c0

    print(
        f"Input: A={a}, B={b}\n"
        f"Normalized output: {pred_norm:.4f}\n"
        f"Original scale output: {pred_denorm:.4f}\n"
        f"Predicted class: {assigned_class}"
    )
```

```python
def run_experiment(p=8, N=6, lr=0.1, seed=None):
    if seed is not None:
        np.random.seed(seed)

    X_train, y_train, X_test, y_test, x_min, x_max, _, _ = prepare_data(
        a=2.8,
        b=0.24,
        c=1.4,
        d=0.58,
        p=p
    )

    net = NN(
        [p, N, 1],
        [sigmoid, linear],
        [der_sigmoid, der_linear],
        [init_xavier, init_xavier],
        [init_zero_thresholds, init_zero_thresholds]
    )

    net, errors, epochs, converged = train_network(
        net,
        X_train,
        y_train,
        lr=lr
    )

    train_mae, train_rmse, train_smape = evaluate_network(
        net, X_train, y_train, x_min, x_max
    )

    test_mae, test_rmse, test_smape = evaluate_network(
        net, X_test, y_test, x_min, x_max
    )

    return {
        "p": p,
        "N": N,
        "epochs": epochs,
        "converged": converged,
        "final_mse": errors[-1],
        "train_mae": train_mae,
        "train_rmse": train_rmse,
        "train_smape": train_smape,
        "test_mae": test_mae,
        "test_rmse": test_rmse,
        "test_smape": test_smape
    }


def experiment_p_values(p_values=(2, 4, 8, 16), N=6, runs=5):
```

```
    results = []

    for p in p_values:
        print(f"Experiments for p={p}")

        for seed in range(runs):
            result = run_experiment(
                p=p,
                N=N,
                seed=seed
            )
            results.append(result)

            print(
                f"Launch {seed + 1}: "
                f"epochs={result['epochs']}, "
                f"MAE={result['test_mae']:.4f}, "
                f"convergence={result['converged']}"
            )

    return results

def summarize_experiments(results, parameter):
    values = sorted(set(result[parameter] for result in results))

    for value in values:
        group = [
            result for result in results
            if result[parameter] == value
        ]

        epochs = np.array([r["epochs"] for r in group])
        maes = np.array([r["test_mae"] for r in group])
        failed = sum(not r["converged"] for r in group)

        print(f"\n{parameter}={value}")
        print(f"Avg. count of epochs: {np.mean(epochs):.2f}")
        print(f"Disperce epochs (std): {np.std(epochs):.2f}")
        print(f"Avg. test MAE: {np.mean(maes):.4f}")
        print(f"Disperce MAE (std): {np.std(maes):.4f}")
        print(f"Didn't converged: {failed}/{len(group)}")


def experiment_N_values(N_values=(1, 2, 6, 12), p=8, runs=5):
    results = []

    for N in N_values:
        print(f"\nExperiments for N={N}")

        for seed in range(runs):
            result = run_experiment(
```

```
                p=p,
                N=N,
                seed=seed
            )
            results.append(result)

            print(
                f"Launch {seed + 1}: "
                f"epochs={result['epochs']}, "
                f"MAE={result['test_mae']:.4f}, "
                f"convergence={result['converged']}"
            )

    return results


if __name__ == "__main__":
    np.random.seed(12)

    # Variant 12
    a, b, c, d = 2.8, 0.24, 1.4, 0.58
    p, N = 8, 6
    train_size, total_size = 140, 200
    lr = 0.1

    k = np.arange(total_size)
    series = a * np.cos(b * k) + c * np.sin(d * k)

    # Data preparation
    (
        X_train, y_train, X_test, y_test,
        x_min, x_max, train_series, test_series
    ) = prepare_data(a, b, c, d, p, train_size, total_size)

    print("Variant 12")
    print(f"Series length: {total_size}")
    print(f"Training interval: 0..{train_size - 1}")
    print(f"Test interval: {train_size}..{total_size - 1}")
    print(f"Normalization: min={x_min:.4f}, max={x_max:.4f}")
    print(f"Window p={p}, hidden neurons N={N}")
    print(f"Training examples: {len(X_train)}")
    print(f"Test examples: {len(X_test)}")

    # Initial time series plot
    plt.figure(figsize=(12, 5))
    plt.plot(k, series, label="Original series")
    plt.axvline(train_size - 0.5, color="red", linestyle="--",
                label="Train/test boundary")
    plt.xlabel("k")
    plt.ylabel("x(k)")
    plt.title("Generated time series")
```

```
    plt.legend()
    plt.grid(True)
    plt.show()

    # Function for training a network with selected parameters
    def fit_configuration(p_value, N_value, seed=12):
        np.random.seed(seed)

        (
            Xtr, ytr, Xte, yte,
            xmin, xmax, train_raw, test_raw
        ) = prepare_data(
            a, b, c, d,
            p=p_value,
            train_size=train_size,
            total_size=total_size
        )

        network = NN(
            [p_value, N_value, 1],
            [sigmoid, linear],
            [der_sigmoid, der_linear],
            [init_xavier, init_xavier],
            [init_zero_thresholds, init_zero_thresholds]
        )

        network, errors, epochs, converged = train_network(
            network,
            Xtr,
            ytr,
            lr=lr,
            target_error=1e-4,
            max_epochs=20000
        )

        return {
            "net": network,
            "errors": errors,
            "epochs": epochs,
            "converged": converged,
            "p": p_value,
            "N": N_value,
            "X_train": Xtr,
            "y_train": ytr,
            "X_test": Xte,
            "y_test": yte,
            "x_min": xmin,
            "x_max": xmax
        }

    # Baseline experiment
```

```python
baseline = fit_configuration(p, N)

net = baseline["net"]
errors = baseline["errors"]

train_pred_norm = net.forward(X_train)
test_pred_norm = net.forward(X_test)

train_pred = denormalize(train_pred_norm, x_min, x_max)
test_pred = denormalize(test_pred_norm, x_min, x_max)

train_true = denormalize(y_train, x_min, x_max)
test_true = denormalize(y_test, x_min, x_max)

train_mae, train_rmse, train_smape = calculate_metrics(
    train_true, train_pred
)
test_mae, test_rmse, test_smape = calculate_metrics(
    test_true, test_pred
)

print("\nBASELINE EXPERIMENT")
print(f"Configuration: p={p}, N={N}")
print(f"Learning rate: {lr}")
print(f"Epochs: {baseline['epochs']}")
print(f"Converged: {baseline['converged']}")
print(f"Final Es (MSE): {errors[-1]:.8f}")

print("\nTraining metrics (original scale):")
print(f"MAE: {train_mae:.6f}")
print(f"RMSE: {train_rmse:.6f}")
print(f"sMAPE: {train_smape:.4f}%")

print("\nTest metrics (original scale):")
print(f"MAE: {test_mae:.6f}")
print(f"RMSE: {test_rmse:.6f}")
print(f"sMAPE: {test_smape:.4f}%")

# Original series and one-step predictions
train_k = np.arange(p, train_size)
test_k = np.arange(train_size, total_size)

plt.figure(figsize=(14, 6))
plt.plot(k, series, label="Original series", alpha=0.65)
plt.plot(train_k, train_pred.ravel(), label="Train prediction")
plt.plot(test_k, test_pred.ravel(), label="Test prediction")
plt.axvline(train_size - 0.5, color="red", linestyle="--",
            label="Train/test boundary")
plt.xlabel("k")
plt.ylabel("x(k)")
plt.title("One-step-ahead prediction")
```

```
plt.legend()
plt.grid(True)
plt.show()

# Convergence: baseline and two window configurations
window_configs = [
    fit_configuration(2, N, seed=21),
    baseline,
    fit_configuration(16, N, seed=23)
]

plt.figure(figsize=(10, 6))
for result in window_configs:
    plt.plot(
        np.arange(1, len(result["errors"]) + 1),
        result["errors"],
        label=f"p={result['p']}, N={N}"
    )

plt.yscale("log")
plt.xlabel("Epoch")
plt.ylabel("Training MSE")
plt.title("Training convergence")
plt.legend()
plt.grid(True)
plt.show()

# Actual versus predicted values on the test set
plt.figure(figsize=(7, 7))
plt.scatter(test_true.ravel(), test_pred.ravel(), alpha=0.7)

low = min(test_true.min(), test_pred.min())
high = max(test_true.max(), test_pred.max())

plt.plot([low, high], [low, high], "r--", label="y = x")
plt.xlabel("Actual values")
plt.ylabel("Predicted values")
plt.title("Actual vs predicted (test set)")
plt.legend()
plt.grid(True)
plt.show()

# Window size experiment: five runs per configuration
p_results = experiment_p_values(
    p_values=[2, 4, 8, 16],
    N=N,
    runs=5
)
summarize_experiments(p_results, "p")

# Hidden layer size experiment: five runs per configuration
```

```python
N_results = experiment_N_values(
    N_values=[1, 2, 6, 12],
    p=p,
    runs=5
)
summarize_experiments(N_results, "N")

# Select the best configuration by mean test MAE
all_results = p_results + N_results

best_key = min(
    {(r["p"], r["N"]) for r in all_results},
    key=lambda config: np.mean([
        r["test_mae"] for r in all_results
        if (r["p"], r["N"]) == config
    ])
)

best_p, best_N = best_key
best = fit_configuration(best_p, best_N, seed=42)
best_net = best["net"]

print("\nBEST CONFIGURATION")
print(f"p={best_p}, N={best_N}")
print(f"Epochs: {best['epochs']}")
print(f"Converged: {best['converged']}")

# Recursive forecast: 30 steps starting at k=140
history = list(series[:train_size])
forecast = []

for step in range(30):
    window = np.array(history[-best_p:])
    window_norm = normalize(window, x_min, x_max).reshape(1, -1)

    prediction_norm = best_net.forward(window_norm)[0, 0]
    prediction = denormalize(prediction_norm, x_min, x_max)

    forecast.append(prediction)
    history.append(prediction)

forecast = np.array(forecast)
future_k = np.arange(train_size, train_size + 30)
actual_future = (
    a * np.cos(b * future_k)
    + c * np.sin(d * future_k)
)

abs_errors = np.abs(actual_future - forecast)
series_range = series.max() - series.min()
error_limit = 0.05 * series_range
```

```
exceeded = np.where(abs_errors > error_limit)[0]

print("\nRECURSIVE FORECAST")
print(f"Forecast starts at k={train_size}")
print(f"Series range: {series_range:.6f}")
print(f"5% error threshold: {error_limit:.6f}")

if len(exceeded) > 0:
    print(f"First step exceeding 5%: {exceeded[0] + 1}")
    print(f"k={future_k[exceeded[0]]}")
    print(f"Absolute error: {abs_errors[exceeded[0]]:.6f}")
else:
    print("Error does not exceed 5% during 30 steps.")

plt.figure(figsize=(12, 5))
plt.plot(future_k, actual_future, "o-", label="Actual")
plt.plot(future_k, forecast, "o-", label="Recursive forecast")
plt.xlabel("k")
plt.ylabel("x(k)")
plt.title("30-step recursive forecast")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(np.arange(1, 31), abs_errors, "o-", label="Absolute error")
plt.axhline(error_limit, color="red", linestyle="--",
            label="5% of series range")
plt.xlabel("Forecast step")
plt.ylabel("Absolute error")
plt.title("Recursive forecast error")
plt.legend()
plt.grid(True)
plt.show()

# Demonstration on three training and three test windows
print("\nWINDOW DEMONSTRATIONS")

for label, indices in [
    ("Train", [20, 60, 100]),
    ("Test", [145, 165, 185])
]:
    print(f"\n{label} windows:")

    for target_k in indices:
        window = series[target_k - best_p:target_k]
        window_norm = normalize(window, x_min, x_max).reshape(1, -1)

        prediction_norm = best_net.forward(window_norm)[0, 0]
        prediction = denormalize(prediction_norm, x_min, x_max)
```

```
            actual = series[target_k]

            print(
                f"k={target_k}, actual={actual:.4f}, "
                f"predicted={prediction:.4f}, "
                f"absolute error={abs(actual - prediction):.4f}"
            )

    # User-defined window

    # USER WINDOW
    print("\nUSER WINDOW")
    print(f"Enter {best_p} consecutive values separated by spaces.")
    print("The values must be in the original scale.")
    print("Enter 'q' to exit.")

    while True:
        user_input = input("Window: ").strip()

        if user_input.lower() == "q":
            print("Interactive mode finished.")
            break

        try:
            values = [float(v) for v in user_input.split()]

            if len(values) != best_p:
                print(f"Expected exactly {best_p} values, got {len(values)}.")
                continue

            if not np.all(np.isfinite(values)):
                print("Error: values must be finite numbers.")
                continue

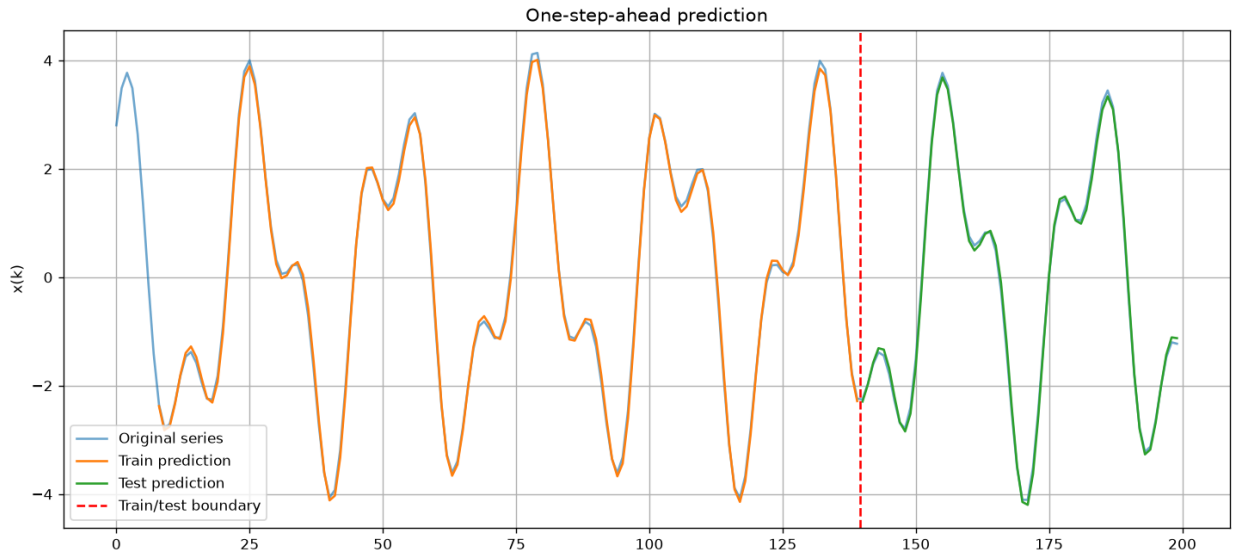
            # Normalize using training data min and max
            user_window = np.array(values, dtype=float)
            user_window_norm = (user_window - x_min) / (x_max - x_min)

            # Predict the next value
            prediction_norm = best_net.forward(user_window_norm)[0]
            prediction = prediction_norm * (x_max - x_min) + x_min

            print(f"Input values: {user_window}")
            print(f"Normalized input: {user_window_norm}")
            print(f"Predicted next value: {prediction:.6f}")

        except ValueError:
            print("Invalid input. Enter numbers separated by spaces, or 'q' to
exit.")
```

4. Результаты базового эксперимента: число эпох, Es, метрики на обучающем и тестовом участках.



Configuration: p=8, N=6
Learning rate: 0.1
Epochs: 144
Converged: True
Final Es (MSE): 0.00009519

Training metrics (original scale):
MAE: 0.068844
RMSE: 0.080114
sMAPE: 12.1728%

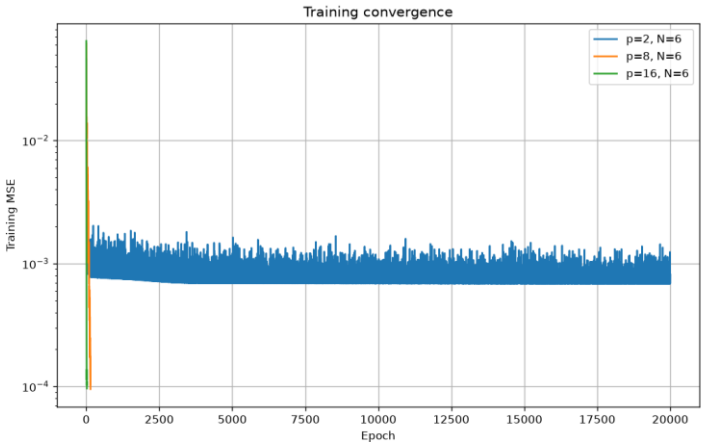
Test metrics (original scale):
MAE: 0.069323
RMSE: 0.079088
sMAPE: 6.7317%

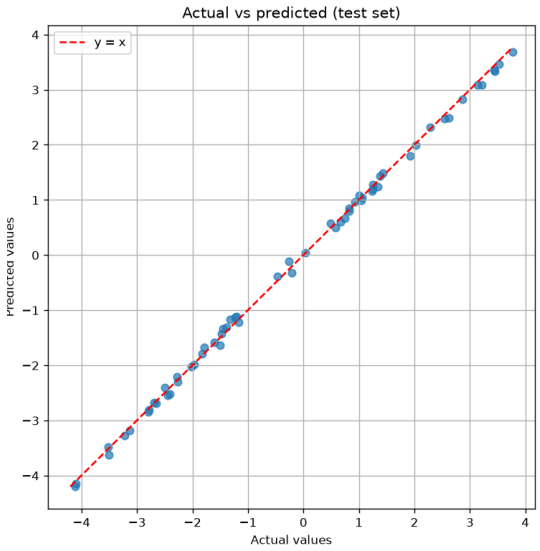
5. Таблицы исследований по p и по N (5 запусков, среднее и разброс).

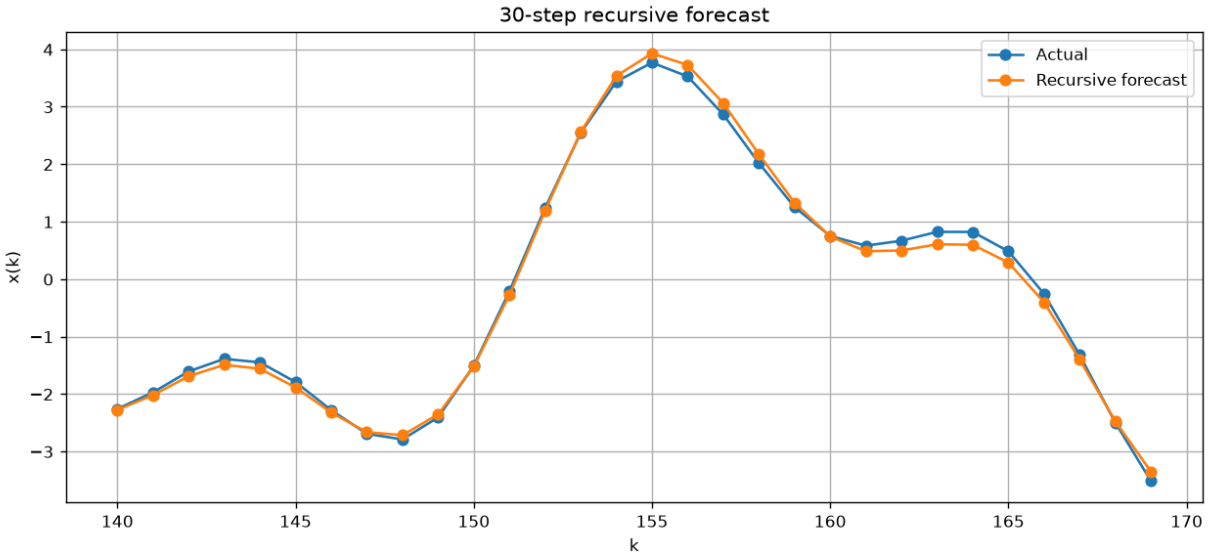
Параметр p	Ср. кол-во эпох	Std (эпохи)	Ср. Test MAE	Std (MAE)	Сходимость
2	20 000.00	0.00	0.1939	0.0069	0из5
4	7 002.80	393.65	0.0688	0.0008	5из5
8	151.80	9.50	0.0697	0.0011	5из5
16	29.00	9.65	0.061	0.0047	5из5

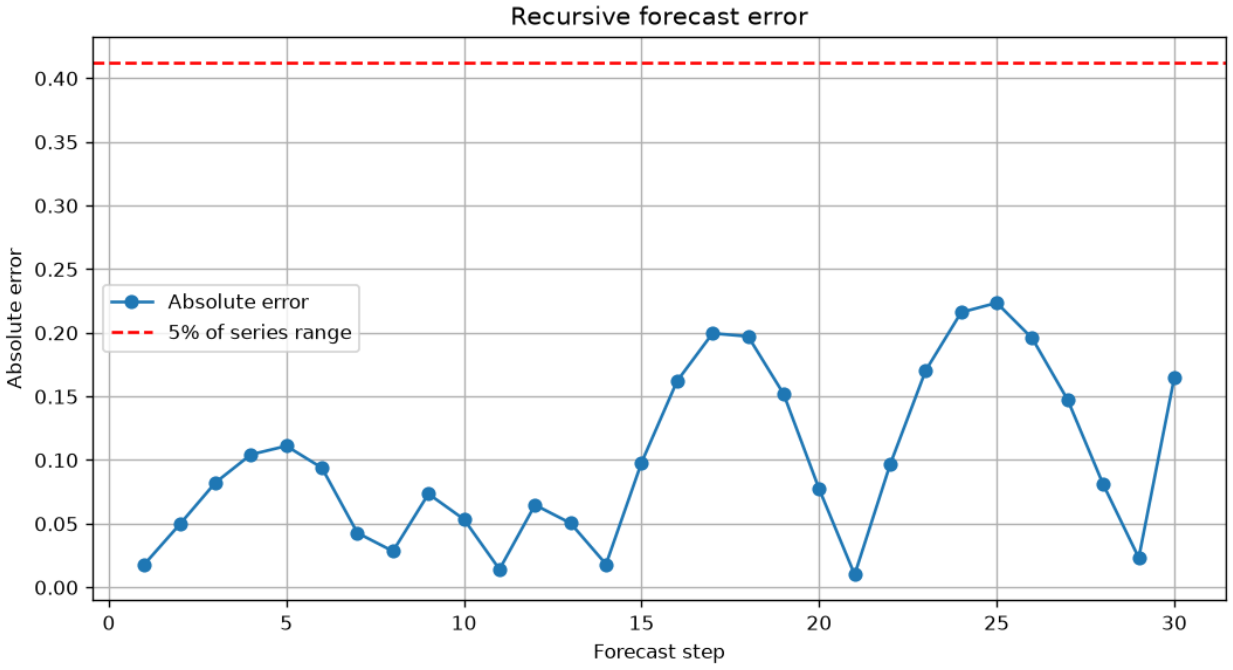
Параметр N	Ср. кол-во эпох	Std (эпохи)	Ср. Test MAE	Std (MAE)	Сходимость
1	163.40	25.07	0.0665	0.0017	5из5
2	159.80	15.10	0.0678	0.0017	5из5
6	151.80	9.50	0.0697	0.0011	5из5
12	130.20	9.99	0.0688	0.0032	5из5

6.


Training convergence
p=2, N=6
p=8, N=6
p=16, N=6
Training MSE
Epoch


Actual vs predicted (test set)
y = x
Predicted values
Actual values

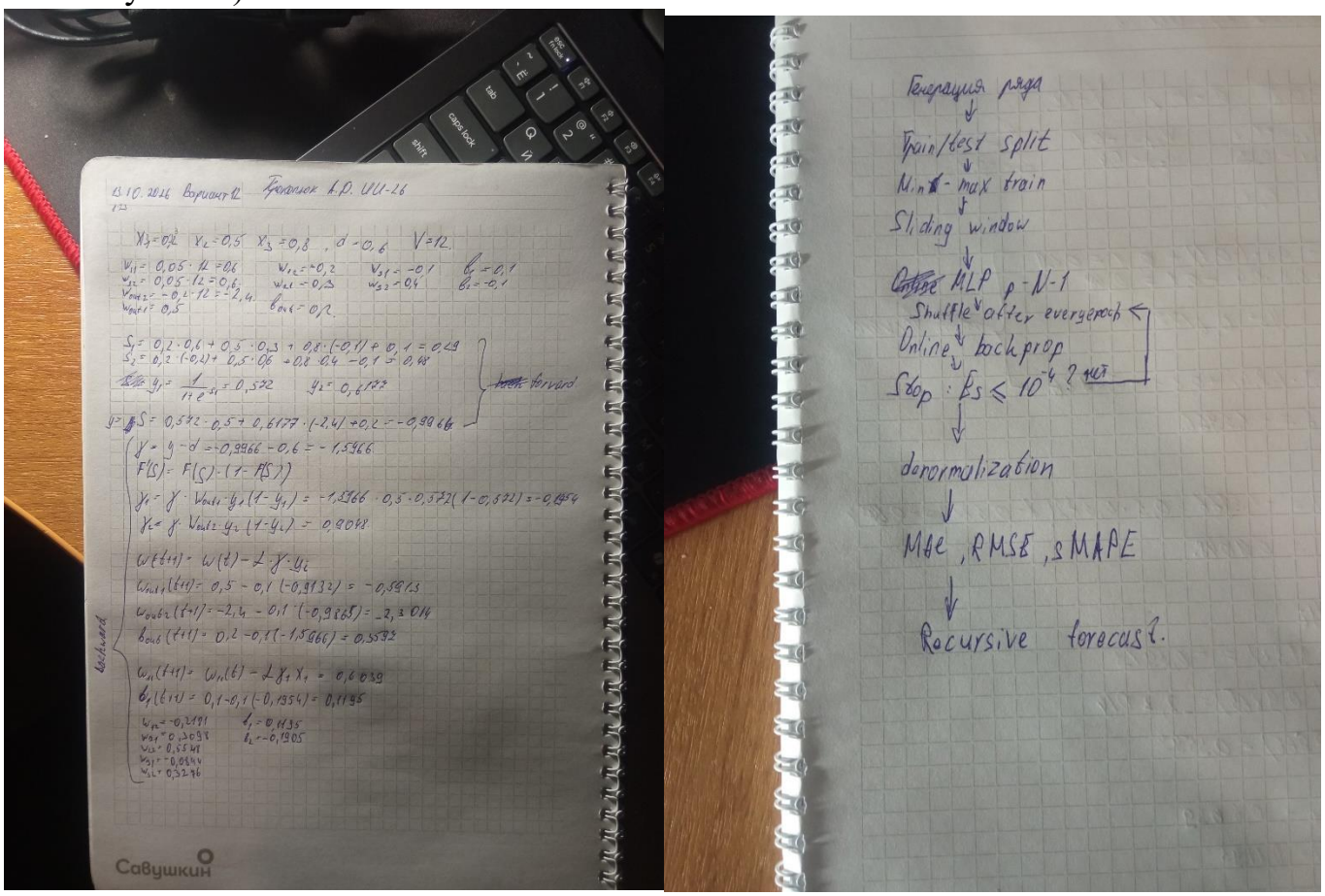

30-step recursive forecast
Actual
Recursive forecast
x(k)
k


Recursive forecast error
Absolute error
5% of series range
Absolute error
Forecast step

7. Результаты работы сети в режиме функционирования (не менее 7–8 примеров).

```
USER WINDOW
Enter 16 consecutive values separated by spaces.
The values must be in the original scale.
Enter 'q' to exit.
Window: -0.9070 -0.8124 -0.6351 -0.3912 -0.1054 0.1958 0.4872 0.7441 0.9435 1.0662 1.0998 1.0405 0.8941 0.6754 0.4061
0.1132
Input values: [-0.907  -0.8124 -0.6351 -0.3912 -0.1054  0.1958  0.4872  0.7441  0.9435
  1.0662  1.0998  1.0405  0.8941  0.6754  0.4061  0.1132]
Normalized input: [0.38617792 0.39769842 0.41929023 0.44899266 0.48379773 0.52047823
 0.55596527 0.58725086 0.61153403 0.62647658 0.63056843 0.6233468
 0.60551803 0.57888449 0.54608881 0.5104191 ]
Predicted next value: 0.080325
Window: -1.0636 -1.2431 -1.3802 -1.4667 -1.4973 -1.4712 -1.3920 -1.2671 -1.1070 -0.9248 -0.7350 -0.5528 -0.3933 -
0.2700 -0.1942 -0.1735
Input values: [-1.0636 -1.2431 -1.3802 -1.4667 -1.4973 -1.4712 -1.392  -1.2671 -1.107
 -0.9248 -0.735  -0.5528 -0.3933 -0.27   -0.1942 -0.1735]
Normalized input: [0.36710699 0.34524726 0.32855106 0.31801699 0.31429048 0.31746897
 0.32711404 0.34232451 0.36182168 0.38401022 0.40712429 0.42931282
 0.44873692 0.46375254 0.47298356 0.47550443]
Predicted next value: 1.126459
Window: 2.5777 2.8201 3.0250 3.1782 3.2690 3.2910 3.2431 3.1291 2.9575 2.7411 2.4964 2.2426 2.0003 1.7903 1.6322
1.5434
Input values: [2.5777 2.8201 3.025  3.1782 3.269  3.291  3.2431 3.1291 2.9575 2.7411
 2.4964 2.2426 2.0003 1.7903 1.6322 1.5434]
Normalized input: [0.81054886 0.84006862 0.86502159 0.88367846 0.8947362  0.89741538
 0.89158206 0.87769901 0.85680135 0.8304479  0.80064805 0.76973998
 0.74023239 0.71465834 0.69540474 0.68459057]
Predicted next value: -2.515146
Window: 0.4826 0.8041 1.1123 1.3865 1.6070 1.7578 1.8280 1.8120 1.7112 1.5335 1.2935 1.0112 0.7110 0.4182 0.1580 -
0.0475
Input values: [ 0.4826  0.8041  1.1123  1.3865  1.607   1.7578  1.828   1.812   1.7112
  1.5335  1.2935  1.0112  0.711   0.4182  0.158  -0.0475]
Normalized input: [0.55540507 0.59455773 0.63209069 0.6654831  0.69233585 0.71070045
 0.71924949 0.71730099 0.70502545 0.68338493 0.65415744 0.61977861
 0.5832199  0.54756236 0.5158749  0.49084886]
Predicted next value: -1.264938
Window: 3.2130 3.3501 3.4050 3.3731 3.2562 3.0620 2.8031 2.4964 2.1621 1.8228 1.5025 1.2248 1.0103 0.8741 0.8250
0.8612
Input values: [3.213  3.3501 3.405  3.3731 3.2562 3.062  2.8031 2.4964 2.1621 1.8228
 1.5025 1.2248 1.0103 0.8741 0.825  0.8612]
Normalized input: [0.88791645 0.90461265 0.91129844 0.90741362 0.8931774  0.86952749
 0.83799834 0.80064805 0.75993659 0.71861623 0.67960971 0.64579108
 0.61966901 0.60308241 0.59710295 0.60151143]
Predicted next value: -2.746941
Window: -2.1040 -1.9275 -1.6552 -1.3104 -0.9220 -0.5221 -0.1430 0.1850 0.4361 0.5912 0.6401 0.5830 0.4312 0.2054 -
0.0642 -0.3510
Input values: [-2.104  -1.9275 -1.6552 -1.3104 -0.922  -0.5221 -0.143   0.185   0.4361
  0.5912  0.6401  0.583   0.4312  0.2054 -0.0642 -0.351 ]
Normalized input: [0.24040583 0.26190021 0.29506123 0.33705139 0.3843512  0.4330515
 0.47921876 0.51916299 0.54974225 0.56863051 0.57458561 0.5676319
 0.54914552 0.52164733 0.48881511 0.45388827]
Predicted next value: 0.940844
```

8. Фото/скан рукописных расчётов (графическая схема пайплайна, ручной расчёт 1 шага обучения).



9.Вывод с ответами на вопросы ниже.

Вывод: на основе многослойного персептрона p–N–1 решили задачу прогнозирования временного ряда. Научился формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Вопросы для вывода

1. Как размер окна p влияет на качество прогноза? При каком p ошибка перестаёт уменьшаться? Почему слишком маленькое окно (p = 2) не позволяет точно описать ряд из двух гармоник, а слишком большое не даёт выигрыша?

Размер окна определяет, сколько предыдущих значений используется для прогноза. При слишком малом окне сеть не получает достаточно информации для описания двух гармоник, а увеличение окна сверх определённого значения не даёт существенного улучшения, но усложняет обучение. Оптимальное значение определяется экспериментально по ошибке на тестовой выборке.

2. Как число скрытых нейронов N влияет на точность и скорость обучения? Наблюдается ли переобучение (малая ошибка на обучении при заметно большей на тесте)? Достаточно ли минимального N, чтобы воспроизвести сумму двух гармоник?

Увеличение числа скрытых нейронов повышает возможности сети, но увеличивает время обучения и риск переобучения. Минимального числа нейронов может быть недостаточно для точного воспроизведения суммы двух гармоник. Оптимальное значение выбирается по соотношению точности и скорости обучения.

3. Почему на выходе для задачи прогнозирования используется линейная функция, а не сигмоида (как в ЛР №3)? Что произошло бы при сигмоиде на выходе и при обучении на ненормализованных значениях?

На выходе используется линейная функция, поскольку прогнозируемые значения могут выходить за диапазон [0;1]. Сигмоида ограничила бы результат этим диапазоном и затруднила прогнозирование. При обучении на ненормализованных значениях сигмоида также могла бы насыщаться, из-за чего обучение замедлилось бы.

4. Почему нормализация должна опираться только на обучающий участок? К чему приводит выход тестовых значений за диапазон [0; 1] и как это проявляется на ваших данных?

Нормализация выполняется по обучающему участку, чтобы не использовать информацию из тестовых данных. Если тестовые значения выходят за обучающий диапазон, их нормализованные значения могут быть меньше 0 или больше 1. Это допустимо для линейного выхода, но может затруднять прогнозирование, особенно если сеть не встречала подобных значений при обучении.

5. Насколько устойчиво обучение к случайной инициализации весов (по сериям из 5 запусков)? Были ли запуски, не достигшие Ee за 20 000 эпох, и от чего это зависит (η, N, p)?

Устойчивость обучения оценивается по пяти запускам с разными начальными весами. Различия в ошибке и числе эпох возникают из-за случайной инициализации. Если некоторые запуски не достигли заданной ошибки за 20 000 эпох, это может быть связано с параметрами обучения, размером сети и окна, а также с неудачной инициализацией.

6. Как растёт ошибка рекурсивного многошагового прогноза с увеличением горизонта? Объясните накопление ошибки и сравните одношаговый прогноз с многошаговым. До какого горизонта прогноз можно считать пригодным?

При рекурсивном прогнозировании ошибка обычно накапливается: предсказанное значение становится входом для следующего шага, поэтому неточности предыдущих прогнозов влияют на последующие. Одношаговый прогноз использует реальные предыдущие значения, поэтому обычно точнее многошагового. Пригодный горизонт определяется экспериментально по моменту, когда абсолютная ошибка превышает 5% диапазона ряда.

7. Как согласуется различие между ошибкой на обучающем и тестовом участках с понятием обобщающей способности? Какие меры (ранняя остановка, уменьшение N, добавление шума, увеличение обучающей выборки) могли бы улучшить обобщение? Что показал эксперимент с шумом?

Разница между ошибками на обучающей и тестовой выборках характеризует обобщающую способность сети. Если тестовая ошибка заметно выше, возможно переобучение. Улучшить обобщение могут ранняя остановка, уменьшение числа нейронов и увеличение обучающей выборки. Добавление шума иногда повышает устойчивость сети, но его эффект необходимо подтверждать экспериментально; без результатов сравнения нельзя утверждать, что в данной работе шум улучшил прогноз.